

Project 2: Algorithm Design and Analysis

DP + Greedy + DAC

Gavin Wilson + Stefan Dedic + Cameron Goz

Section 1: Introduction

This project explores three major algorithm design paradigms: Divide and Conquer, Greedy, and Dynamic Programming. Each paradigm is applied to a classic problem that demonstrates how different strategies are used to solve problems efficiently.

For DAC, we test merge sort, which breaks a large sorting problem into smaller sub problems, solves them recursively, and combines results. For greedy, we analyze the activity selection problem, where the goal is to choose the max number of nonoverlapping activities using the locally optimal strategy. For dynamic programming, we look at the 0/1 knapsack problem, where the objective is to maximize value w/o exceeding a weight limit by reusing solutions to overlapping subproblems.

Section 2: Divide and Conquer

1. Explanation:

- o What is the problem?

The problem is to sort a list of numbers into an ascending order. Given an unsorted array, rearrange the elements so each is less than or equal to the one after it.

- o What is the approach?

The approach uses merge sort. The array is repeatedly split into two smaller halves until each subarray contains only one element or is empty. Since arrays of size 0 and 1 are already sorted, the algorithm treats it as the base case and merges those small, sorted subarrays back together in order. During merging, it compares the front elements of the two halves and places the smaller one into a new merged array. This continues until the entire original array is rebuilt.

2. Pseudocode: step by step logic

MergeSort(array):

if length of array ≤ 1 :

return array

mid = length of array // 2

Left half = MergeSort(array from start to mid)

Right half = MergeSort(array from mid to end)

return Merge(left half, right half)

Merge(left, right):

create empty list merged

set $i = 0$

set $j = 0$

while $i < \text{length of left}$ and $j < \text{length of right}$:

if $\text{left}[i] \leq \text{right}[j]$:

append $\text{left}[i]$ to merged

$i = i + 1$

else:

append $\text{right}[j]$ to merged

$j = j + 1$

while $i < \text{length of left}$:

append $\text{left}[i]$ to merged

$i = i + 1$

while $j < \text{length of right}$:

append $\text{right}[j]$ to merged

$j = j + 1$

return merged

3. Why this paradigm works

Merge sort works as a divide and conquer paradigm because it breaks one large sorting problem into smaller subproblems of the same type. Each half is sorted recursively, and then the two sorted halves are combined using merge. This works well because sorting smaller arrays is easier, and once two halves are already sorted, they can be merged in linear time. The algorithm keeps dividing until it reaches naturally sorted arrays (base cases 0 and 1), then combining them to form the final result.

Complexity Analysis

Time Complexity:

- Best Case: $O(n \log n)$ -> $\log n$ divisions, and n time for merge
- Worst Case: $O(n \log n)$

Space Complexity: $O(n)$ -> merged array

Section 3: Greedy

4. Explanation:

- o What is the problem?

The Activity Selection Problem involves selecting the number of non-overlapping activities from a list. Each one has a start and a finish time with the goal of picking as many activities as possible there is to have no overlap in time.

- o What is the approach?

The approach to efficiently solves this problem is to

1. Sort all the activities by finish time in increasing order
2. Select the first activity
3. Iterate through the remaining activities
4. Repeat these steps until all activities have been checked

5. Pseudocode: step by step logic

activity_selection(activities):

If activities is empty:

return []

sort the activities by finish time

Selected = the first activity

Last_finish_time = finish time of first activity

For each activity in the remaining activities:

If activity start time $\geq$ last finish time

Add activity to selected

Last finish time = activity finish time

return selected_activities

6. Why this paradigm works

This problem works well with the greedy paradigm because making the locally optimal choice, selecting the activity that finishes the earliest, leads to a globally optimal solution. By choosing the earliest compatible activity, the algorithm leaves as much time as possible for remaining activities and repeating will guarantee maximum nonoverlapping activities will be selected.

Complexity Analysis

Time Complexity:

- Best Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Space Complexity: $O(n)$

Section 4: Dynamic Programming

7. Explanation:

- o What is the problem?

The Knapsack problem is selecting a subset of items to place into a given “knapsack” such that the total value is maximized without exceeding a given weight capacity. Additionally, each item has an associated weight and value, and each item can be included or excluded (0/1).

- o What is the approach?

The approach uses Dynamic Programming with memoization to efficiently solve the problem. At each item, there are two possible decisions: include the item if it fits within the remaining capacity, or exclude it and move to the next item. We recursively evaluate both choices and take the maximum value. Since many subproblems are repeated, such as repeated indexes and remaining capacity, we store the results in a memoization table to avoid recomputation.

8. Pseudocode: step by step logic

DP(weights, values, capacity, index):

if index >= number of items OR capacity == 0:
 return 0

if (index, capacity) exists in memo:
 return memo[(index, capacity)]

Option 1: skip the item
not_take = DP(weights, values, capacity, index + 1)

Option 2: take the item (if it fits)
take = 0
if weights[index] <= capacity:
 take = values[index] + DP(weights, values, capacity - weights[index], index + 1)

memo[(index, capacity)] = max(take, not_take)

return memo[(index, capacity)]

9. Why this paradigm works

Dynamic Programming is effective for the 0/1 Knapsack problem because it has overlapping subproblems, same item index and remaining capacity that are solved multiple times in a recursive approach, and optimal substructure which means the problem can be constructed using optimal solutions of smaller problems.

Complexity Analysis

Time Complexity:

- Best Case: $O(n * \text{Capacity})$
- Worst Case: $O(n * \text{Capacity})$

Space Complexity: $O(n * \text{Capacity})$

Section 5: Results

Merge Sort Results:

```
Test: Small random array
Input:  [5, 2, 9, 1, 3]
Output: [1, 2, 3, 5, 9]
Time:   0.006200 ms
-----
Test: Already sorted array
Input:  [1, 2, 3, 4, 5, 6, 7]
Output: [1, 2, 3, 4, 5, 6, 7]
Time:   0.009000 ms
-----
Test: Reverse sorted array
Input:  [7, 6, 5, 4, 3, 2, 1]
Output: [1, 2, 3, 4, 5, 6, 7]
Time:   0.005800 ms
-----
Test: Empty array
Input:  []
Output: []
Time:   0.000800 ms
-----
Test: Single element
Input:  [42]
Output: [42]
Time:   0.000500 ms
-----
Test: Medium random array
Input:  [179, 362, 168, 702, 794, 752, 350, 207, 963, 499, 137, 314, 945, 938, 354, 147, 738, 16, 444, 86, 319, 475, 432, 150, 276]
Output: [16, 86, 137, 147, 150, 168, 179, 207, 276, 314, 319, 350, 354, 362, 432, 444, 475, 499, 702, 738, 752, 794, 938, 945, 963]
Time:   0.016000 ms
-----
```

Activity Selection Problem Results:

Test Case 1: Small input size

Input Activities:

('A1', 1, 4)
('A2', 3, 5)
('A3', 0, 6)
('A4', 5, 7)
('A5', 8, 9)
('A6', 5, 9)

Selected Activities:

('A1', 1, 4)
('A4', 5, 7)
('A5', 8, 9)

Total activities selected: 3

Execution time: 0.007400ms

Test Case 2: Medium input size

Input Activities:

('A1', 1, 2)
('A2', 3, 4)
('A3', 0, 6)
('A4', 5, 7)
('A5', 8, 9)
('A6', 5, 9)
('A7', 6, 10)
('A8', 8, 11)
('A9', 2, 13)
('A10', 12, 14)

Selected Activities:

('A1', 1, 2)
('A2', 3, 4)
('A4', 5, 7)
('A5', 8, 9)
('A10', 12, 14)

Total activities selected: 5

Execution time: 0.005400ms

Test Case 3: Edge case (empty input)

Input Activities:

Selected Activities:

Total activities selected: 0

Execution time: 0.000700ms

0/1 Knapsack Results:

Test 1 (Small): 55

Execution time: 0.009200 ms

Test 2 (Medium): 7

Execution time: 0.008900 ms

Test 3 (Edge - Empty): 0

Execution time: 0.000400 ms

Section 6: Conclusion

After analyzing the three different algorithm paradigms, we learned multiple things. Merge sort demonstrated how DAC simplifies large problems by recursion, breaking them into smaller pieces to then combine results. Activity selection showed how greedy paradigms can produce optimal solutions solely by focusing on the local best option. Knapsack illustrated how DP can efficiently solve optimization problems by storing solutions to subproblems that get repeated.

Overall, this project strengthened our understanding of algorithm design and how to approach them. It displayed the importance of matching a problem to an efficient solution.